AIEPROGRAM | WEEK7

MAINTAINER'S COPILOT

A fine-tuned classifier, advanced RAG, an authenticated chatbot with memory, embeddable in any host app, and evals that fail your CI. Solo. Five days.

Week 7 Project — Individual

Your repo. Your code. You answer every question on Friday.

THE MISSION

■ DEADLINE

End of week — submission Friday — 10-minute presentation

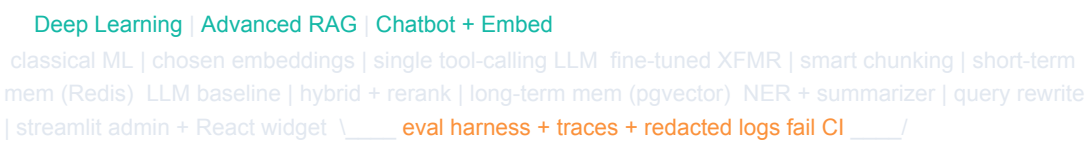
Build a **Maintainer's Copilot**: an authenticated chatbot an open-source maintainer talks to when triaging issues. It classifies issues into bug / feature / docs / question using three models you compare on the same test set. It extracts entities, summarizes threads, and answers maintainer questions using advanced RAG over the project's docs and resolved issues. It carries memory across conversations. It is **embeddable as a real production shaped widget** — a small standalone React bundle that any host app can drop in — and you will demo it running in a host app you build. The things that matter — the classifier, the retrieval, the generation — have golden evals that fail CI when they regress.

This week ran two tracks: **deep learning for NLP** (text processing and representations, fine-tuning transformers, NER / classification / summarization pipelines, comparing ML vs DL vs LLM) and **LLM engineering** (advanced RAG, RAG and LLM evaluation, chatbots with tools and memory, observability and safe logging). Both are in this project.

The architecture is graded. Secrets in Vault. Blob in MinIO. Layers respected. Traces visible. Logs redacted. Exceptions handled. Same standard as week 6.

Solo project. Five days. Don't add scope.

ARCHITECTURE



SUGGESTED 5-DAY SCHEDULE

Each day assumes ~6 focused hours. The chatbot day is the heaviest; plan accordingly.

MON	**Foundations**	Repo skeleton, `docker-compose` with all services, Vault wired, tracing wired from day one, Alembic baseline, dataset fetch, splits. Start fine-tuning.
TUE	**DL track**	Finish classifier + model card. Classical ML and LLM baselines. Three-way comparison. Classification golden set. NER and summarization endpoints.
WED	**Advanced RAG**	Corpus, embedding choice, chunking, hybrid retrieval, reranking, one query technique. RAG golden set. Redaction layer + exception handling refactor.
THU	**Chatbot + memory + embed**	Auth, Streamlit chatbot, tools, short- and long-term memory, widget config, React widget bundle, loader script, one host app. Both eval suites in CI. Ship.
FRI AM	**Polish + present**	Final integration, CI green, READMEs done, practice. Demo in the afternoon.

DATASET

Closed issues from one open-source repo of your choice. Pick it Monday morning and live with it.

- **Classification labels** come from maintainer-applied labels, mapped to bug / feature / docs / question. Define the mapping in `DECISIONS.md`.
- **Splits** stratified, test strictly more recent in time than train.

- **RAG corpus** is the project's docs plus a held-out slice of resolved issues with maintainer answers. Held-out issues do *not* appear in classifier training.
- **Golden sets:** 25 examples each for classification and RAG, hand-curated.

THE DEEP LEARNING TRACK

TEXT PROCESSING & REPRESENTATIONS

- A real preprocessing pipeline for the issue corpus. Defend your choices in `DECISIONS.md`.
- An embedding model choice for the RAG corpus, backed by a retrieval-quality number against at least one alternative on your golden set.

FINE-TUNING A TRANSFORMER

- Fine-tune a small encoder for issue classification. Track training with a real run logger. Save the artifact with a model card listing architecture, hyperparameters, training data hash, and final metrics.
- Document and defend your freeze policy.

NLP PIPELINES AS TOOLS

- An NER tool extracting code-shaped entities from issue text. Integration only.
- A summarization tool. Pre-trained or LLM-driven.
- Both behind FastAPI endpoints the chatbot calls over HTTP.

ML VS DL VS LLM

- A classical ML baseline on the same splits.
- An LLM baseline on the same test split.
- Three-way comparison in `DECISIONS.md`: accuracy, macro-F1, per-class F1, latency, cost. Defend a deployment choice.

ADVANCED RAG

Naive fixed-size chunking + pure-dense retrieval is the baseline you beat. Every choice off the baseline is justified with a number on your golden set.

- A chunking strategy that is not naive fixed-size.
- Hybrid retrieval combining sparse and dense, with a tuned weighting.
- Cross-encoder reranking over the top-k from hybrid.
- A query transformation technique.
- Metadata filtering over the corpus.

- Vector store: `pgvector` or `Qdrant`.

EVALUATION

Two golden sets. Two CI gates. Committed thresholds in `eval_thresholds.yaml`.

CLASSIFICATION EVAL

- 25-issue hand-curated set, separate from the test split.
- Macro-F1, per-class F1, confusion matrix. Run against all three models.

RAG EVAL

- 25 question / ideal-answer / ground-truth-chunks triples.
- Retrieval metrics and generation metrics. RAGAS or a frozen judge model — your choice.
- Hand-label 5 of the 25 yourself. Report agreement with the judge.

Both suites run in CI on every push. `eval_report.json` is written every run, stored in blob, and diffed against the previous green build. Regression below threshold blocks merge.

THE CHATBOT

A single tool-calling LLM. Not a workflow, not a multi-agent system — one LLM that picks tools.

AUTHENTICATION

- `fastapi-users` with JWT. Email and password registration.
- JWT signing key resolves from Vault at startup.
- Two roles: `user` and `admin`. Admin can invite users and configure widgets.

TOOLS

- Tools that wrap *your* classifier, NER, summarizer, and RAG pipeline.
- An explicit `write_memory` tool. No auto-writes.
- Prompts as files in `prompts/`, version-controlled.

MEMORY

- **Short-term** conversation state in Redis. TTLs are explicit and justified.
- **Long-term** memory in Postgres with pgvector. At least one of {episodic, semantic, procedural}. Defend the choice.
  - Every long-term write produces an audit-log row: actor, action, target, timestamp.

- Cross-conversation recall demo on Friday.

EMBEDDABLE WIDGET

The chatbot has two surfaces: a Streamlit app for authenticated users and admins (login, full chat, memory inspector, widget configuration) and a **standalone React widget** that host apps embed. The widget is the production-shaped surface; the Streamlit app is the internal tool. They share one backend API.

WHY TWO FRONTENDS

- Internal tools want fast iteration. Streamlit is good at that. The admin config page, the memory inspector, the authenticated chat — all Streamlit.
- Embedded widgets want small bundles, fast first paint, real `postMessage`, and host adaptive styling. Streamlit-in-iframe is the wrong tool. The widget is React.
- Both call the same FastAPI backend. The widget does not know or care that Streamlit exists.

THE REACT WIDGET

- A small standalone React app, built with Vite (or equivalent), output to a single bundled JS file.
- The bundle is served from the API or from MinIO with proper cache headers. Its size is in the submission block — keep it lean.
- Chat panel, input box, streamed messages, a collapsed bubble that expands. Tailwind or vanilla CSS — your call.

- One `postMessage` channel between widget and host — at minimum for iframe resize. - Theme (primary color, position) comes from the widget config at runtime, not hardcoded.

WIDGET CONFIGURATION

- A `widget` table in Postgres, keyed by public `widget_id`. Fields: allowed origins (list), theme, greeting, enabled tools.
- An admin-only configuration page in the Streamlit app where admins create and edit widget configs and see the generated embed snippet for each.

EMBED FLOW

-

A loader script served from `/widget.js`. The host pastes a single `<script>` tag with their `data-widget-id` and the loader injects the iframe pointing at the React widget bundle.

- The widget reads its config at load time and styles itself accordingly.
- A `demo/host/` folder with one host page (tiny static-server container) that embeds the widget. Friday demo runs the widget in this host.

ORIGIN ALLOWLISTING (THE PRODUCTION PRACTICE)

- CORS allowlist is enforced from the widget's `allowed_origins` field in the database, **not** from a hardcoded env var.
- The embed route sets a `Content-Security-Policy` header with `frame-ancestors` matching the widget's allowed origins. Unallowed parents cannot iframe the widget; the browser blocks the embed.
- Friday demo: show the widget loading on an allowed host, then show it blocked on a host whose origin is not in the allowlist. Both demos use real browser network and console output.

OBSERVABILITY & SAFE LOGGING

From week 6 session 4. All three apply to every service.

TRACING

- Pick a tracing backend and defend the choice in `DECISIONS.md`.
- Every LLM call, tool call, and RAG retrieval is a span. A conversation is a trace tree rooted at the user message.
- Span attributes include model name, token counts, latency, and tool inputs / outputs *after redaction*.
- The trace ID is logged alongside every structured log line for the same request, so logs and traces are joinable.
- Friday demo: open the tracing UI and walk through a real conversation's trace tree, including one trace that hit an error path.

REDACTION

- A redaction layer runs before any log line, trace span, or memory write leaves the service boundary.

You define the patterns. Defend the list in `SECURITY.md`. Think hard about what shows up in real issue text and what shouldn't show up in your logs.

- Tested explicitly. A test asserts that a message containing a fake API key never appears unredacted in logs, traces, or memory.
- Redaction is in the `app/infra/` layer and used by every service. **EXCEPTION HANDLING**

- A domain exception hierarchy (e.g. `NotFoundError`, `PermissionDenied`, `ToolFailure`) distinct from infrastructure exceptions.
- Mapped to HTTP responses at the API boundary via a single exception handler. Users never see a stack trace; they see a structured error with a code and a request ID.

- Tool failures inside the chatbot are caught and recovered. If the classifier endpoint is down, the chatbot says so and falls back; it does not 500.
- Every uncaught exception is logged with the trace ID and the request ID.

ENGINEERING & ARCHITECTURE

The codebase is layered. Same standard as week 6.

- `app/api/` for HTTP only. Routers do not touch SQLAlchemy, Redis, or external systems.
- `app/services/` owns business logic, transaction boundaries, cache and memory invalidation.
- `app/repositories/` owns SQL. No HTTP errors. No cache invalidation.
- `app/domain/` holds Pydantic domain models, distinct from SQLAlchemy ORM models.
- `app/infra/` holds adapters for Vault, MinIO, Redis, LLM providers, the model server, the tracing backend, and the redaction layer.
- The boundary will be checked on Friday by asking you to add a new endpoint or tool live. **SECRETS**

- Every secret resolves from Vault at startup. LLM API keys, tracing backend keys, JWT signing key, database password, MinIO credentials.
- `.env` holds only the Vault root token and ports.
- `grep -ri 'sk-' app/` and `grep -ri 'password' app/` return zero matches outside Vault reading code.

- The app refuses to boot if Vault is unreachable. **BLOB**

& DATABASE

- MinIO holds: model artifacts (or a manifest), `eval_report.json` from every CI run, training plots, and per-conversation retrieved-chunks snapshots for the last N conversations.
- Postgres 16 with pgvector. Schema in Alembic migrations. A `migrate` container runs `alembic upgrade head` and exits before `api` boots.
- Audit log table for role changes, memory writes, widget config changes, conversation deletions.

REFUSE TO BOOT

- `api` refuses to boot if Vault is unreachable, classifier weights are missing, the weights' SHA-256 does not match the model card, the tracing backend is misconfigured, or any committed eval threshold is set to zero / disabled.

COMPOSE STACK

api FastAPI app (auth, chat, memory, RAG orchestration, widget config). **chatbot** Streamlit app

(auth UI, admin config, memory inspector, full chat). **widget** Static server for the built React widget

bundle and the loader script.

model server FastAPI inference server for classifier, NER, summarizer.

host nginx serving the demo host app.

migrate Alembic entrypoint, exits.

db `postgres:16` with pgvector.

redis `redis:7` , short-term memory and cache.

minio `minio/minio` , blob.

vault `hashicorp/vault` , dev mode.

`docker-compose up` from a fresh clone after `cp .env.example .env` and filling in the Vault root token. CI on every push: lint, type-check, build images, run both eval suites against golden sets, run the redaction test, smoke-test the stack.

THINK ABOUT

Three models, three numbers, one production. Which one ships, and does the answer survive a change in scale, latency budget, or failure cost?

How do you know your embedding model is right for *this* corpus rather than the benchmark it was advertised on?

The LLM-as-judge disagrees with you on a hand-labeled sample. Who's right, how do you know, and what do you do with the judge in CI after that?

Short-term memory in Redis with a TTL. What's the TTL, why that number, and what happens at the boundary?

Your widget is 180KB gzipped. The host site's product manager says that's too big. What can you cut, and what's the cost of cutting it? At what bundle size does the conversation reverse and you push back instead?

A user pastes a stack trace into the chat that contains their GitHub token. Where could that string end up if your redaction misses it, and how would you find out before they did?

Your trace UI shows a span took 4.3 seconds and you don't know why. What's missing from the trace, and why is that a design decision, not an accident?

Vault becomes unreachable. The app is already running. What happens, what *should* happen, and where does the policy live?

These are your problems to solve. No hints.

SUBMISSION

Public GitHub repo, tag `v0.1.0-week7`, comes up cleanly with `docker-compose up` from a fresh clone after `cp .env.example .env`.

Project 7 - [Name]
Repo: [GitHub URL]
Tag: v0.1.0-week7
Dataset: [chosen repo] issues, [N train / N val / N test]
Classification — Classical: F1=[n] | Fine-tuned: F1=[n] | LLM: F1=[n]
Deployment choice: [model] - because [one line]
Embedding model: [name] - chosen because [one line]
RAG — hit@5=[n] | MRR@10=[n] | Faithfulness=[n] | Answer relevancy=[n]
Long-term memory type: [episodic | semantic | procedural]
Tracing backend: [name] - chosen because [one line]
Widget bundle size: [n] KB (gzipped)
LLM: [provider + model]
README contains: ARCH.md, DECISIONS.md, RUNBOOK.md, EVALS.md, SECURITY.md

RULES

■ NO VIBE CODING

Understand every line you ship. You will be asked about it on Friday.

■ THE ARCHITECTURE IS THE GRADE

A working chatbot in a tangled codebase scores below a slightly-worse chatbot in a clean one. Layers respected, secrets in Vault, blob in MinIO, traces visible, logs redacted, exceptions handled.

■ THE EVALS ARE THE GRADE

A great-looking chatbot without working evals scores below a worse-looking one whose CI fails when you regress. Committed thresholds. They mean something.

■ EVERY DECISION IS BACKED BY A NUMBER

Embedding model, chunking strategy, deployment choice, retrieval weighting — every choice in `DECISIONS.md` is backed by a number on your golden set.

■ LOGS ARE REDACTED, TRACES ARE REAL

A redaction test proves it. A trace tree demo proves the rest.

Ship it.

END OF WEEK .